

COOPERATIVE ROBOTICS

Authors:

Lovecchio Marco
Marmolino Giorgio
Petrosilli Marco

Email:

s7647356@studenti.unige.it
s7721100@studenti.unige.it
s7812048@studenti.unige.it

Date: 29/01/2026

General notes

- Exercise 1 is done with the ROBUST matlab main and unity visualization tools. Exercises 2-3 are done with the Franka simulation tools.
- Comment and discuss the simulations, in a concise scientific manner. Further comments, other than the questions, can be added, although there is no need to write 10 pages for each exercise.
- Aid the discussion with screenshots of the simulated environment (compress them to maintain a small overall file size), and graphs of the relevant variables (i.e. activation functions of inequality tasks, task variables, and so on). Graphs should always report the units of measure in both axes, and legends whenever relevant.
- You can add vertical bars to plots to highlight periods of time (e.g. when one action is active), or vertical ones to highlight thresholds (e.g. the minimum altitude value).
- Report the thresholds whenever relevant.
- Report the mathematical formula employed to derive the task jacobians and the control laws when asked, including where they are projected.
- If needed, part of the code can be inserted as a discussion reference.
- Provide your answers in the subsection where the question is made.

Use the following template when you need to discuss the hierarchy of tasks of a given action or set of actions. Please report the tasks in the same order as in the unified hierarchy implemented in your code.

Table 1: Example of actions/hierarchy table: a number in a given cell represents the priority of the control task (row) in the hierarchy of the control action (column). The type column indicates whether the objective is an equality (E) or inequality (I) one.

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
Task name A	I	1		1
Task name B	I	2	1	
Task name C	E		2	2

1 Exercise 1: Implement a Complete Mission

Implement several actions to reach a point, land, and then perform the manipulation of a target object.

Initialize the vehicle in a position that allows you to test all the tasks. Then, use a "safe waypoint navigation action" to reach the following position:

$$\begin{bmatrix} 10.5 & 37.5 & -38 & 0 & -0.06 & 0.5 \end{bmatrix}^\top$$

Then land, aligning to the nodule. In particular, the x axis of the vehicle should align to the projection, on the inertial horizontal plane, of the unit vector joining the vehicle frame to the nodule frame. Once landed, implement a "fixed-based manipulation action" to reach the target nodule (mimicking the scanning of the nodule). During this manipulation phase, the vehicle should not move for any reason. Considering that the goal position for the vehicle could be not so precise with respect to the position of the nodule, implement a solution that guarantees that, once landed, the nodule is certainly within the manipulator's workspace. If needed, you can change also the vehicle's goal position if you need to stress the system and force certain tasks to activate (e.g., the distance to the nodule one).

1.1 Q1: Report the unified hierarchy of tasks used, the actions and their tasks, specifying their priorities in each action.

1.1.1 Task Hierarchy and Action Configuration

The mission execution relies on a carefully designed task hierarchy distributed across four distinct actions, each tailored to specific operational phases. The task hierarchy adheres to a structured priority framework where safety-critical tasks occupy the highest priority levels to ensure collision avoidance and system integrity, followed by operational prerequisite tasks that establish necessary conditions for mission execution such as workspace verification, vehicle immobility, and alignment requirements, then action-defining tasks that characterize the specific operational objective of each mission phase, and finally optimization tasks that enhance performance metrics such as manipulability and dexterity within the available null space.

1.1.2 Unified Task Hierarchy

The specific ordering of tasks within each action reflects a deliberate control architecture designed to ensure mission safety, operational feasibility, and performance optimization in a cascading priority structure. **Safety-critical tasks** occupy the highest priority to provide absolute guarantees: **Minimum Safety Altitude** prevents seabed collision, while **Manipulator Joint Lock** ensures arm immobility during vehicle-dominated phases, overriding all other objectives when thresholds are approached. **Operational prerequisite tasks** follow immediately, establishing the necessary conditions for mission execution: **Manipulability Check** verifies workspace accessibility, **Horizontal Attitude** maintains orientation stability, **Target Alignment** establishing the required geometrical approach, while **Maintain Position (X,Y)** enforce vehicle immobility during manipulation. **Action-oriented tasks**, positioned at low-intermediate priority levels, characterize the specific objective of each phase: **Landing Altitude** and **Pose Control** execute the navigation goal and landing, while **Tool Control** drives the end-effector to the nodule in $\mathcal{A}_4$. These tasks operate within the null space of higher-priority constraints, meaning they are satisfied only insofar as they do not violate safety or prerequisite conditions. **Optimization tasks**, such as **Dexterity Objective** and **Look Ahead**, occupy the lowest priority levels, activating only when the available null space permits workspace verification and configuration improvement without compromising primary objectives.

This hierarchical structure ensures that if conflicts arise between tasks, the iCAT algorithm automatically sacrifices lower-priority objectives to satisfy higher-priority constraints, resulting in robust and predictable system behavior throughout all mission phases. Considering the complete mission execution and the sequential activation of actions, the unified task hierarchy across all phases is presented in order of maximum priority level achieved:

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$	$\mathcal{A}_4$	$\mathcal{U}$
Minimum Safety Altitude	I	1	1			1
Manipulator Joint Lock (home position)	E	2	2	1		2
Horizontal Attitude	I	3	3		1	3
Target Alignment	E		4	3	2	4
Look Ahead	E	4				5
Manipulability Check	I		5	4	3	6
Maintain Position (X,Y)	E			5	4	7
Landing Altitude	E			6	5	8
Pose Control	E	5				9
Dexterity Objective	I				6	10
Tool Control	E				7	11

The unified task hierarchy exhibits several design features that reflect the adaptive nature of the mission control strategy. Several tasks demonstrate priority evolution across actions, with Horizontal Attitude progressing from priority 3 to 2 to 1, and Target Alignment rising from priority 4 to 2, reflecting how their relative importance increases as the mission transitions from navigation to precision manipulation phases where orientation stability becomes critical. Task mutual exclusivity is evident in the action-specific objectives: Look Ahead and Pose Control appear only in $\mathcal{A}_1$ for navigation, while Tool Control appears exclusively in $\mathcal{A}_4$ for manipulation, reflecting operationally incompatible

or strictly sequential objectives that cannot coexist. Prerequisite task persistence ensures robustness throughout the mission. Tasks such as Target Alignment and Horizontal Attitude, once introduced, are maintained active in all subsequent actions with progressively increasing priority, providing continuous disturbance rejection as operational requirements become more stringent. The safety-to-performance transition is exemplified by the Minimum Safety Altitude task, which disappears after $\mathcal{A}_2$ as the vehicle intentionally descends to the seabed during landing in $\mathcal{A}_3$, demonstrating context-aware adaptation of safety constraints to operational requirements. Finally, computational efficiency is achieved through selective task activation: while the unified hierarchy contains 11 distinct tasks, no single action employs more than 6 simultaneously, balancing control completeness with computational tractability and real-time performance requirements.

Here is the tasks list explained:

1. Minimum Safety Altitude (Inequality): Maintains minimum vertical clearance from the seabed to prevent collisions. Activates only when the vehicle approaches the minimum threshold.
2. Manipulator Joint Lock (Equality): Enforces zero velocity on all manipulator joints to keep the arm immobilized when manipulation is not required. Prevents unwanted motion in the null space of vehicle-related tasks.
3. Horizontal Attitude (Inequality): Regulates roll and pitch angles to maintain horizontal orientation, ensuring stable operations and predictable vehicle dynamics. Activates only when attitude deviation exceeds acceptable bounds.
4. Target Alignment (Equality): Aligns the vehicle's longitudinal axis with the horizontal projection of the direction toward the nodule, satisfying the mission requirement for proper approach orientation.
5. Look Ahead (Equality): Orients the vehicle's forward axis toward the navigation waypoint. Incorporates distance-dependent activation that disengages the task within 3-4 meters of the target to prevent conflicts with precise pose alignment.
6. Manipulability Check (Inequality): Monitors and maintains adequate workspace margin by adjusting the vehicle's position when necessary. Activates only when the nodule approaches the workspace boundary.
7. Maintain Position (X,Y) (Equality): Locks the vehicle's horizontal position to prevent lateral drift during descent and manipulation, strictly enforcing zero horizontal velocity.
8. Landing Altitude (Equality): Controls the vehicle's depth to execute the landing maneuver, precisely regulating vertical position to the target landing altitude.
9. Pose Control (Equality): Drives the vehicle's complete 6-DOF pose toward the desired target configuration during navigation, constituting the primary navigation objective.
10. Dexterity Objective (Inequality): Optimizes the manipulator's joint configuration to maximize dexterity during end-effector motion. Activates when dexterity drops below acceptable levels.
11. Tool Control (Equality): Drives the end-effector to the nodule position in 3D space, executing the primary manipulation objective of reaching and scanning the target.

1.2 Q2: Comment the behaviour of the robot, supported by relevant plots. For example, show the altitude and the altitude activation to show that when the altitude becomes too small, the tasks gets activated and a minimum value is guaranteed.

The robot's behavior throughout the mission demonstrates the effectiveness of the hierarchical task-priority control architecture and the smooth activation mechanisms of the iCAT algorithm. This section presents a comprehensive analysis supported by simulation data, illustrating how individual tasks activate, interact, and contribute to successful mission execution across all four operational phases.

1.2.1 Minimum Safety Altitude Task

Figure below presents the vehicle's altitude profile and the corresponding task activation function throughout Actions $\mathcal{A}_1$ and $\mathcal{A}_2$. During the initial navigation phase, the vehicle maintains adequate clearance from the seabed, resulting in near-zero activation as the inequality constraint is not violated. As the vehicle approaches the landing zone and begins descending toward the operational altitude, the distance to the seabed decreases, triggering smooth activation of the safety task. The activation function exhibits the characteristic bell-shaped response, progressively increasing as the altitude approaches the minimum threshold. Once activated, the task successfully prevents further descent below the safety limit, demonstrating absolute priority enforcement over lower-priority navigation objectives. The discontinuation of this task in Action $\mathcal{A}_3$ is intentional and clearly visible, as the vehicle deliberately lands on the seabed where maintaining minimum altitude would conflict with the landing objective.

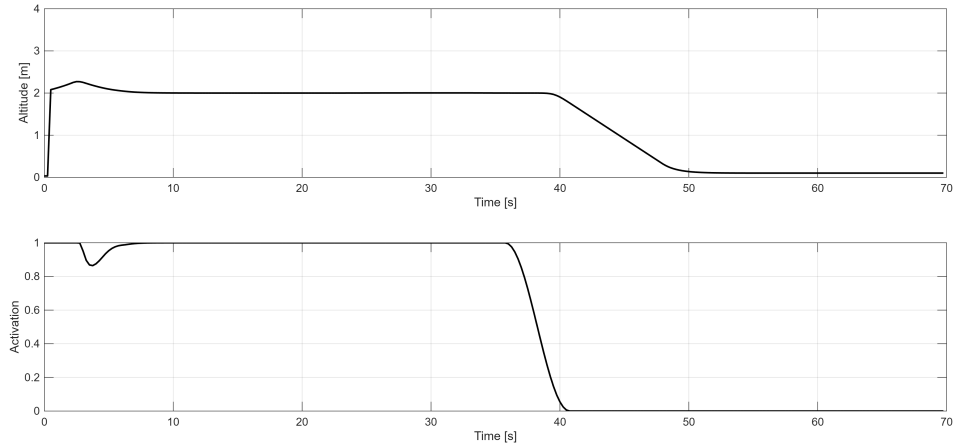


Figure 1: Robot Altitude

1.2.2 Target Alignment Task

Figure below illustrates the evolution of the yaw alignment error and the vehicle's heading convergence toward the nodule direction. The task is first introduced in Action $\mathcal{A}_2$ (Manipulability Check), where the yaw error begins converging toward zero as the vehicle reorients itself to face the target. The alignment behavior persists through Actions $\mathcal{A}_3$ (Landing) and $\mathcal{A}_4$ (Manipulation), with the error maintained near zero (despite possible and not simulated environmental disturbances and dynamic coupling effects). The constant unit activation ($a = 1$) ensures continuous correction authority, providing robust disturbance rejection throughout the descent and manipulation phases. Small oscillations in the error signal during manipulation would reflect the reaction forces from arm motion, which are successfully compensated by the active alignment control operating at priority 2 in $\mathcal{A}_4$.

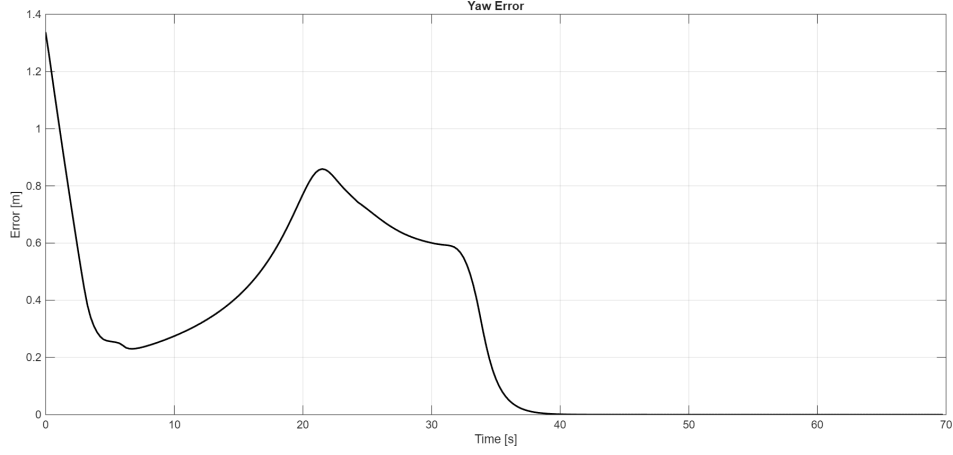


Figure 2: Target alignment: error decrease over time

1.2.3 Manipulability Check Task

Figure below shows the horizontal distance from the vehicle to the nodule compared against the workspace boundary defined by the manipulator’s maximum reach plus base offset. The task activation is computed as the product of two components: a distance-based activation that increases when the workspace margin becomes insufficient, and an angular-based activation that preferentially engages when the vehicle is well-aligned with the target. During Action $\mathcal{A}_2$, when the initial vehicle position places the nodule near or beyond the workspace boundary, the task activates and drives the vehicle closer to the target through horizontal position corrections. The dual activation mechanism is clearly visible: the task exhibits maximum activation only when both the distance error is significant and the vehicle orientation is favorable, preventing destabilizing lateral motions when alignment is poor and allowing the robot to use the main thrust given by the 4 motors, moving in a more efficient way. Once adequate workspace margin is established, the activation decreases and the task operates passively, monitoring without active intervention.

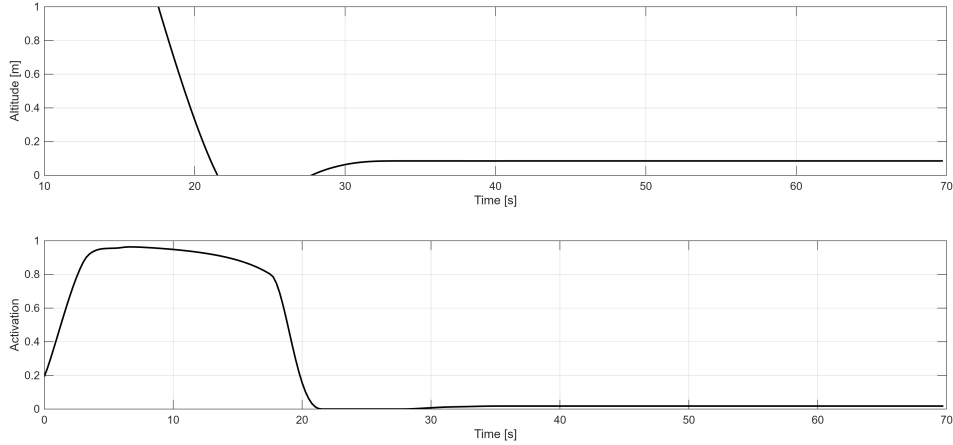


Figure 3: Manipulability error decreasing over time and reaching near zero value during the task activation.

1.2.4 Look Ahead Task

Figure below demonstrates the distance-dependent activation behavior of the Look Ahead task during Action $\mathcal{A}_1$. The task maintains near-unity activation during the majority of the transit phase, continuously orienting the vehicle toward the waypoint to facilitate efficient navigation. As the vehicle enters the final approach zone (approximately 3-4 meters from the target), the activation smoothly de-

creases according to the bell-shaped function, progressively disengaging the look-ahead behavior. This design prevents control conflicts between waypoint-oriented heading and the precise pose alignment required for subsequent actions, allowing higher-priority tasks to dominate orientation control as the operational zone is reached. The smooth deactivation ensures continuity in the commanded velocities, avoiding discontinuous control inputs.

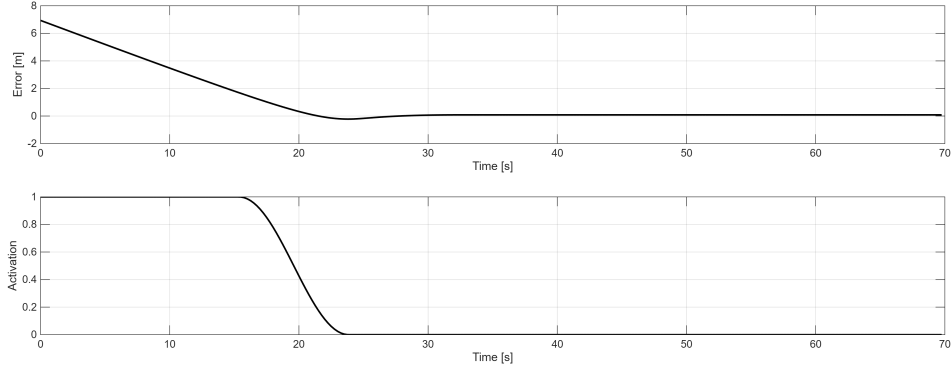


Figure 4: Activation Decreasing when distance is below 3 m

1.2.5 End-Effector Positioning and Vehicle Immobility

Figure below presents the end-effector position error during time. The Tool Control task, operating at priority 6 within the null space of all vehicle immobility constraints, successfully drives the end-effector toward the nodule target with the error converging asymptotically toward zero. The end-effector motion is achieved purely through manipulator joint velocities, with no vehicle drift or rotation (given that simulation disturb reaction forces generated by arm motion are null).

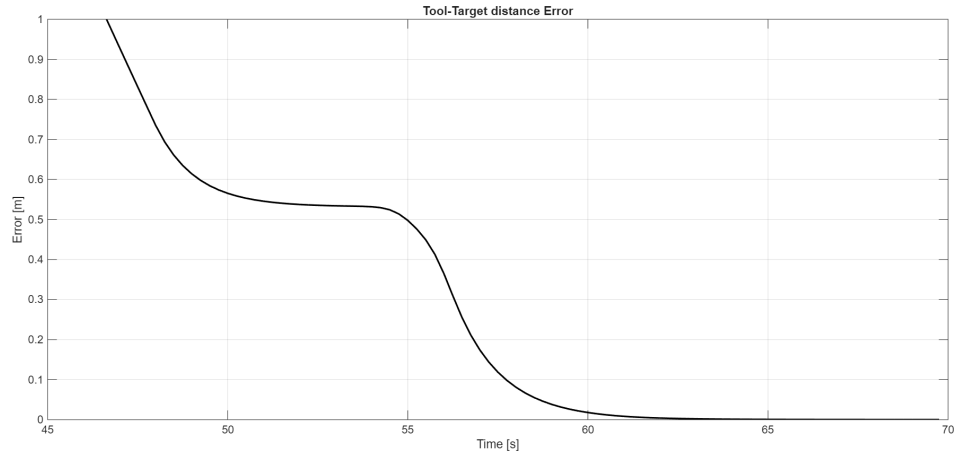


Figure 5: Caption

1.2.6 Mission Trajectory Overview

Figure below provides a three-dimensional visualization of the complete vehicle trajectory . The path clearly delineates the four operational phases: the initial navigation segment toward the waypoint in $\mathcal{A}_1$, the positioning and alignment adjustment in $\mathcal{A}_2$, the controlled descent in $\mathcal{A}_3$, and the stationary manipulation phase in $\mathcal{A}_4$. The trajectory demonstrates smooth transitions between actions with no abrupt discontinuities, confirming that the task activation functions and priority reordering occur without inducing control instabilities.

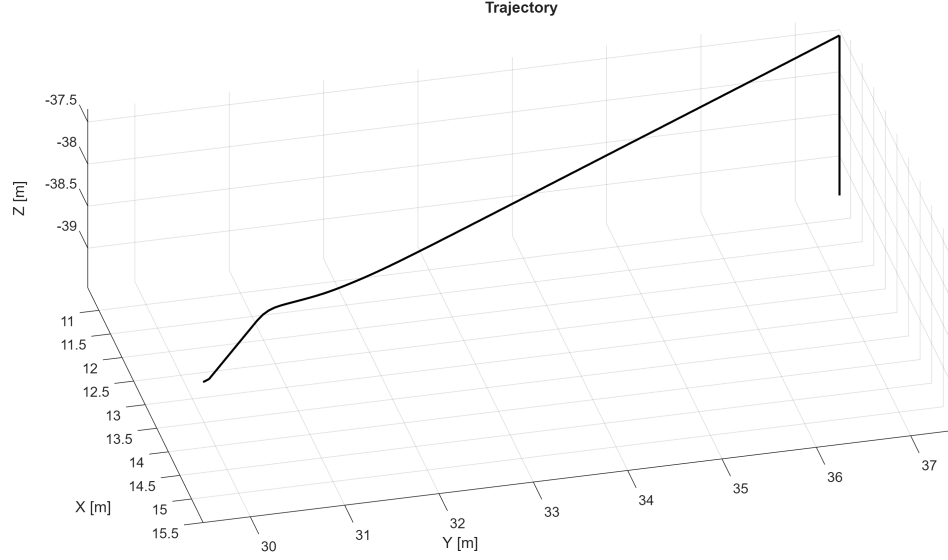


Figure 6: 3D Trajectory

1.3 Q3: What is the Jacobian relationship for the Alignment to Target control task? How was the task reference computed? - UNITÀ MISURA SU GRAFICO

The target alignment task enforces the mission requirement that the vehicle's longitudinal axis aligns with the horizontal projection of the direction toward the nodule. The task reference is computed dynamically based on the current geometric relationship between the vehicle and the target. First, the linear position error vector $\mathbf{e}_{\text{lin}} \in \mathbb{R}^3$ between the target frame $\{g\}$ and the vehicle frame $\{v\}$ is extracted from the homogeneous transformation matrices. The desired yaw angle ψ_{target} is then determined as:

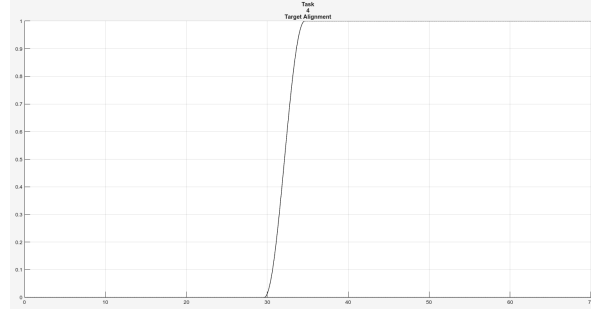
$$\psi_{\text{target}} = \text{atan2}(e_{\text{lin},y}, e_{\text{lin},x})$$

where $e_{\text{lin},x}$ and $e_{\text{lin},y}$ are the horizontal components of the position error. This angle represents the heading required for the vehicle's x-axis to point toward the nodule's horizontal projection. A target rotation matrix $R_{\text{target}} \in SO(3)$ is constructed with this yaw while maintaining zero roll and pitch (horizontal attitude), and combined with the vehicle's current position to form a desired homogeneous transformation ${}^wT_{\text{target}}$. The angular error $\theta_{\text{err}} \in \mathbb{R}^3$ between this target orientation and the vehicle's actual orientation is computed using the Cartesian error function, yielding the task variable $x = \theta_{\text{err}}$. The task reference velocity is set as $\dot{x}_d = 0.8 \cdot \theta_{\text{err}}$, implementing proportional feedback with gain $k_p = 0.8$, and saturated to a maximum magnitude of 0.3 rad/s to prevent aggressive rotational maneuvers.

The task Jacobian $J \in \mathbb{R}^{3 \times 13}$ maps the UVMS velocity vector (7 manipulator joint velocities + 6 vehicle velocities) to the rate of change of the angular error:

$$J = \begin{bmatrix} 0_{3 \times 7} & 0_{3 \times 3} & {}^wR_v \end{bmatrix}$$

where ${}^wR_v \in SO(3)$ is the rotation matrix from the vehicle frame to the world frame. This structure reflects that: (1) manipulator joint velocities do not affect the vehicle's orientation, resulting in a zero block for the first 7 columns; (2) linear vehicle velocities do not contribute to angular motion, yielding a zero block for columns 8-10; and (3) only the vehicle's angular velocities directly influence the orientation error, with the rotation matrix transforming vehicle-frame angular velocities to the world frame where the error is expressed. The Target Alignment task employs constant unit activation



throughout its operation, with the activation matrix set as $A = I_{3 \times 3}$, corresponding to a dimensionless activation coefficient $a = 1$ for all three angular error components. This choice reflects the critical nature of maintaining proper orientation alignment once the task becomes active: unlike tasks such as Minimum Safety Altitude or Manipulability Check, which utilize threshold-based activation functions that smoothly engage only when specific conditions are approached, the Target Alignment task does not require progressive activation modulation. The alignment requirement is binary in nature, either the vehicle must be oriented toward the nodule or it must not; partial activation would provide no meaningful benefit while potentially introducing unnecessary complexity in the control law. The constant full activation ensures that whenever the task is included in the action's task set, it contributes its full Jacobian to the null-space projection hierarchy without attenuation, guaranteeing maximum control authority within its assigned priority level. This is particularly important during the Landing and Manipulation actions, where the task operates at relatively high priority (3 and 2, respectively) to actively reject rotational disturbances; a reduced activation coefficient would weaken this disturbance rejection capability precisely when robustness is most critical for mission success.

1.4 Q4: What have you done to guarantee that the nodule is within the manipulator’s workspace?

Ensuring that the nodule remains within the manipulator’s reachable workspace is critical for mission success, as landing at a position from which the target is unreachable would necessitate aborting and repositioning the vehicle. To provide this guarantee, a geometric approach based on kinematic reachability constraints was implemented through the Manipulability Check task.

The fundamental principle underlying this task is a simple geometric inequality: the manipulator can reach the nodule if and only if the horizontal distance from the vehicle frame to the target does not exceed the sum of the manipulator’s maximum reach and the offset distance between the vehicle frame origin and the manipulator base. Formally, let $d_{\text{nodule}} = \|\mathbf{e}_{\text{lin}}(1 : 2)\|$ denote the horizontal distance from the vehicle frame $\{v\}$ to the nodule frame $\{g\}$, where $\mathbf{e}_{\text{lin}} \in \mathbb{R}^3$ is the linear position error vector. Let L_{arm} represent the manipulator’s maximum horizontal reach (the sum of link lengths when fully extended in the horizontal plane), and let d_{base} denote the fixed horizontal offset between the vehicle frame origin and the manipulator’s base frame. The reachability condition is satisfied when:

$$d_{\text{nodule}} \leq L_{\text{arm}} + d_{\text{base}}$$

The Manipulability Check task monitors the workspace margin by computing the distance error:

$$e_{\text{manip}} = d_{\text{nodule}} - (L_{\text{arm}} + d_{\text{base}})$$

When $e_{\text{manip}} > 0$, the nodule lies outside the reachable workspace, indicating that the vehicle must move closer. When $e_{\text{manip}} \leq 0$, the nodule is within reach and the vehicle position is adequate. The task reference is computed as a proportional control law driving the vehicle toward the nodule in the horizontal plane:

$$\dot{\hat{x}} = k_p e_{\text{lin}}(1 : 2)$$

with gain $k_p = 15$ and saturation at 0.2 m/s to prevent aggressive motions. The task Jacobian $J \in \mathbb{R}^{2 \times 13}$ is structured to map only the vehicle’s horizontal linear velocities to the task space:

$$J = \begin{bmatrix} 0_{2 \times 7} & {}^w R_v(1 : 2, 1 : 3) & 0_{2 \times 3} \end{bmatrix}$$

where ${}^w R_v(1 : 2, 1 : 3)$ extracts the first two rows of the vehicle’s rotation matrix, ensuring that only horizontal translation affects the workspace margin. Critically, the task activation is not constant but rather depends on two coupled conditions through a multiplicative activation function:

$$a_{\text{manip}} = a_{\text{dist}}(e_{\text{manip}}) \cdot a_{\text{ang}}(\theta_{\text{err}})$$

The first component, $a_{\text{dist}}(e_{\text{manip}})$, is an increasing bell-shaped function that activates the task when the workspace margin becomes insufficient:

$$a_{\text{dist}} = f_{\uparrow}(0, 1.0, 0, 1, e_{\text{manip}})$$

where $f_{\uparrow}$ denotes an increasing activation function with threshold at 0 meters and transition region of 1.0 meter. This ensures the task engages smoothly as the vehicle approaches the workspace boundary and remains active with maximum intensity when the nodule is unreachable.

The second component, $a_{\text{ang}}(\theta_{\text{err}})$, introduces angular alignment awareness by computing the angular error between the vehicle’s current orientation and the desired heading toward the nodule (identical to the Target Alignment computation). The activation is defined as:

$$a_{\text{ang}} = f_{\downarrow}(0, 3\pi/5, 0, 1, \|\theta_{\text{err}}\|)$$

where $f_{\downarrow}$ is a decreasing bell-shaped function. This component modulates the task to preferentially activate when the vehicle is already well-aligned with the target direction, reducing the magnitude of lateral corrections when the vehicle’s orientation is far from optimal. The multiplicative coupling $a_{\text{manip}} = a_{\text{dist}} \cdot a_{\text{ang}}$ ensures that the task contributes most strongly when both conditions are favorable: the workspace margin is insufficient *and* the vehicle is reasonably aligned toward the target, preventing destabilizing corrections when orientation errors dominate.

Through this design, the Manipulability Check task provides a proactive guarantee that the vehicle will reposition itself during the dedicated Manipulability Check action ($\mathcal{A}_2$) if necessary, and continues monitoring during the Landing action ($\mathcal{A}_3$) to handle any residual positioning errors. By the time the Fixed-Base Manipulation action ($\mathcal{A}_4$) begins, the geometric constraint ensures with certainty that the nodule lies within the manipulator’s workspace, enabling successful task execution.

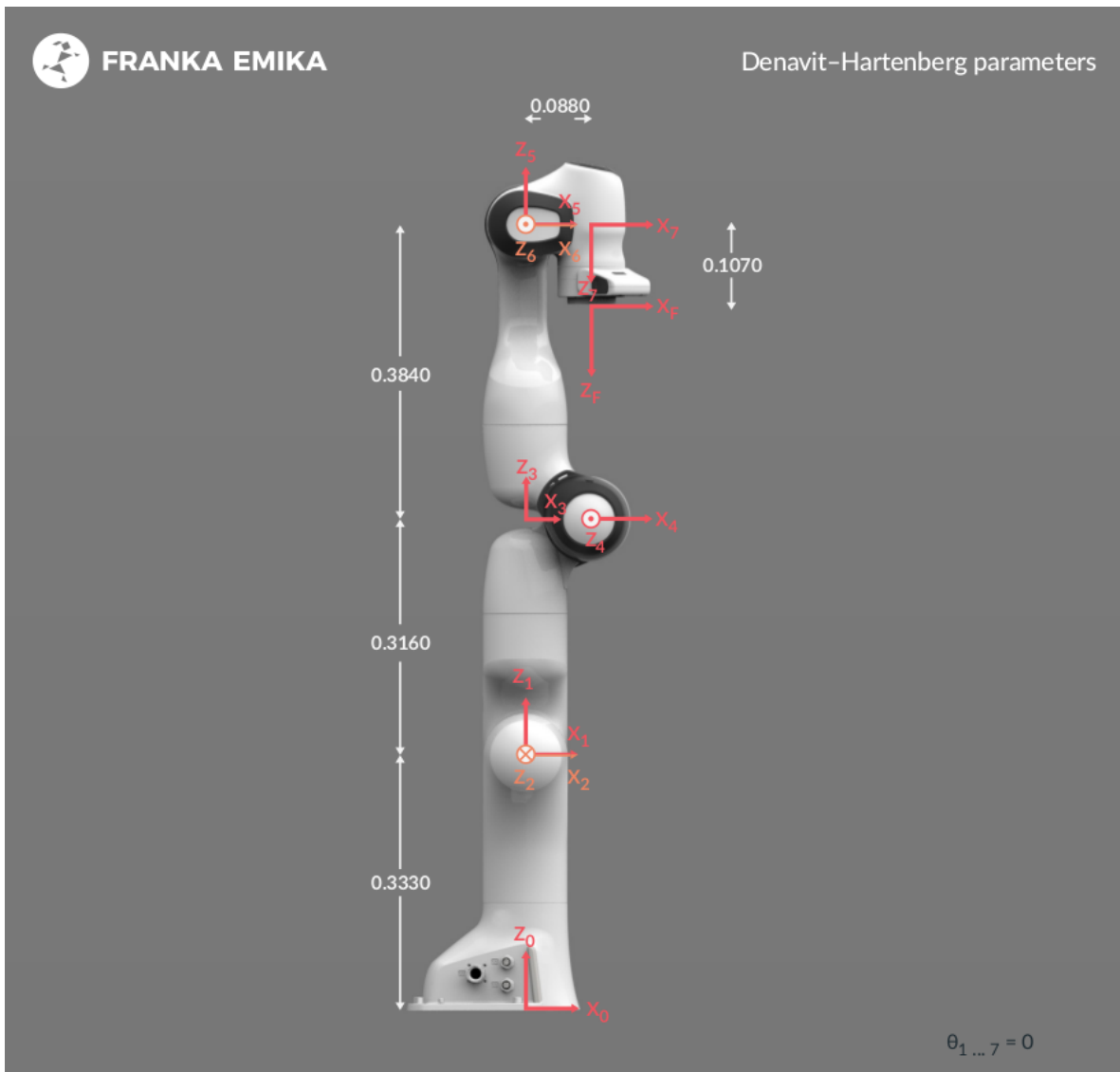


Figure 7: Robot Specifications

2 Exercise 2: Bimanual manipulation

In this exercise it is required to perform a bimanual manipulation by implementing the task priority algorithm considering two manipulators as a single robot. The manipulator adopted for this assignment is the Franka Panda by Emika (see Figure 7)). A simulation in python is provided, in order to visualize the two robots and test your Matlab implementation.

1. The transformations between the world and the base of each robot must be computed knowing that:
 - (a) The left arm base coincides with the world frame.
 - (b) The right arm base is rotated of π w.r.t. z -axis and is positioned 1.06 m along the x -axis and -0.01 m along the y -axis.
2. The transformation from each base to the respective end-effector is given in the code and is retrieved from the URDF model thanks to the *Robotic System Toolbox* of Matlab.
3. Then, define the tool frame for both manipulators; the tool frames must be placed at 21.24 cm along the z -axis of the end-effector frame and rotated with an angle of -44.9949 deg around the z -axis of the end-effector.

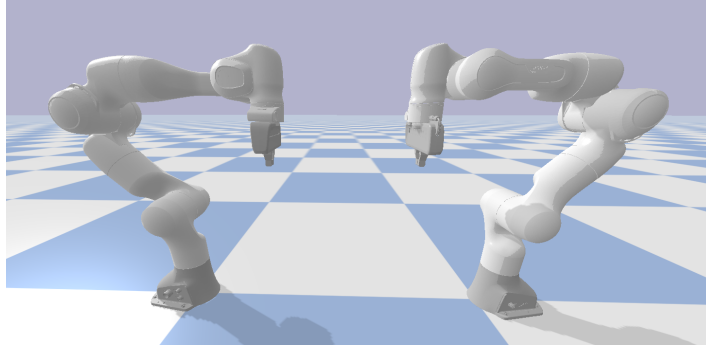


Figure 8: Initial position of the robots

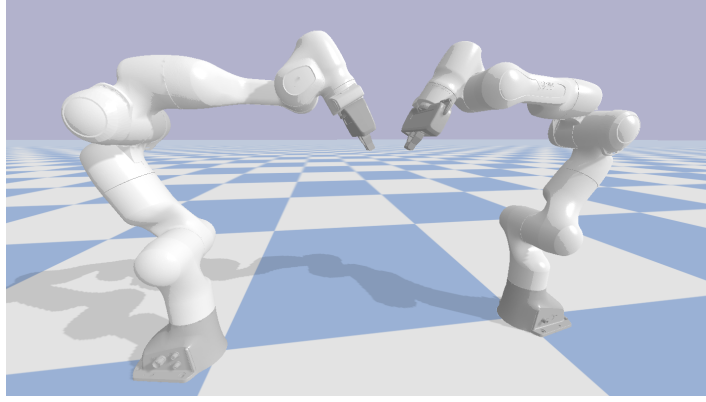


Figure 9: Relative orientation at the grasping position

4. Implement a “move to” action that includes the joint limits task.
5. The first phase foresees to move the tool frame of both manipulators to the grasping points, by implementing a “move to” action and its corresponding tasks. Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT:** the position of the grasping points can be computed by knowing the origin of the object frame wO_o and the object length l_{obj} . The goal orientation of the tool frames is obtained by rotating the tool frames 30 deg around their y-axis; the manipulators should reach the configuration depicted in Figure 9.

$${}^wO_o = [0.5, 0, 0.59]; \quad (1)$$

$$l_{obj} = 10 \text{ (cm)}. \quad (2)$$

6. During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

Once the manipulators reach the grasping points, the second phase of the mission should start. Now, implement the Bimanual Rigid Grasping Constraint task to carry the object as a rigid body.

1. Define the object frame as a rigid body attached to the tool frame of each manipulator. **HINT:** Compute this quantity after reaching the grasping point.
2. Define the rigid grasp task.
3. Then, move the object to another position while both manipulators hold it firmly, e.g. ${}^wO_g = [0.65, -0.35, 0.28]^T (m)$

4. Note that the transition for the *Bimanual Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Repeat the simulation, using a goal position or by changing the grasping in such a way that one of the two manipulators activates multiple safety tasks, to stress the constraint task.

2.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action.

We have `go_to_grasp_set`, `move_grasped_obj_set`, `final_set` and `unified_set`. Each task inside the actions is duplicated, one for the right arm and one for the left arm.

`go_to_grasp_set` is the first action to be performed, in order to grasp the object. It is composed of `joint_limit_task`, which is a safety task, `min_altitude`, also a safety task, and `tool_task`, which is the action-defining task and has the lowest priority. We decided to have the `joint_limit_task` with a higher priority than `min_altitude`, because the joint limits represent hard intrinsic mechanical constraints that must be respected to preserve the integrity of the robot's hardware, while minimum altitude is an environmental constraint that cannot be guaranteed if the system enters an emergency stop state due to mechanical violation.

`move_grasped_obj_set` is the second action, performed when both robots have grasped the object. This set includes `bim_constraint_task`, which is shared between both arms and has the highest priority, `joint_limit_task`, `min_altitude`, and `tool_task_2`.

`final_set` is, as the name suggests, the last action, which only requires `min_altitude` to remain active.

`unified_set` is the actual task set used by the action manager. It contains `bim_constraint_task`, `joint_limit_task`, `min_altitude`, `tool_task`, and `tool_task_2`.

Table 2: Hierarchy table

Task	Type	Attribute	Go to Grasp	Move Grasped Obj	Final	Unified
Bimanual Constraint	E	Constraint		1		1
Joint Limits	I	Safety	1	2		2
Minimum Altitude	I	Safety	2	3	1	3
Tool Task	E	Action Defining	3			4
Tool Task 2	E	Action Defining		4		5

The transition is handled by the shared `StateMachine` of the two arms. By updating the state machine, `dist_to_goal` and `rot_to_goal` of both arms are compared with a threshold.

1. **BIM Constraint Task (Equality):** Enforces the bimanual manipulation constraint by forcing the relative velocities of the tools of both two arms to be zero, considering that the object are rigidly attached to the end-effectors of the arms, while the object is grasped, ensuring coordinated dual-arm motion and preventing internal forces that could damage the object or destabilize the system. Formulated as an equality constraint with the highest priority, this task strictly synchronizes both arms' motion to preserve the grasp geometry and load distribution during cooperative manipulation. This task is shared between both arms, coupling their motion to achieve unified object transport.
2. **Joint Limit Task (Inequality):** Enforces kinematic safety boundaries by preventing manipulator joints from exceeding their mechanical limits during all operational phases. Formulated as an inequality constraint, this task activates only when joints approach their physical bounds, allowing free motion within the safe workspace while intervening to prevent damage or singularities near the limits.

3. **Minimum Altitude (Inequality)**: Maintains a minimum vertical clearance from the table to prevent collisions during dual-arm manipulation and object transport maneuvers, ensuring safe operation throughout all action phases. As an inequality task, it activates only when the vehicle approaches the minimum threshold, allowing greater altitudes without penalization while guaranteeing obstacle avoidance.
4. **Tool Task (Equality)**: Drives the end-effector (tool frame) of each manipulator arm to the object grasp position in 3D space during the initial approach phase, executing the primary objective of reaching and acquiring the target object. The equality constraint ensures precise positioning of each tool at the designated grasp location with minimal error, enabling successful bilateral grasping.
5. **Tool Task 2 (Equality)**: Controls the end-effector trajectories of both manipulator arms to execute the coordinated object transport motion after successful grasping, driving the grasped object along the desired path toward the target location. The equality formulation ensures continuous tracking of the reference trajectory with both arms moving together.

2.2 Q2: What is the Jacobian relationship for the Joint Limits task? How was the task reference computed?

1. Identify Control Variable

In this case the control variable coincides with the joint variables vector.

$$\underline{x} = \underline{q} \in \mathbb{R}^7 \quad (3)$$

2. Objective

We want to guarantee that every joint variable lives within an allowed interval in order to respect physical limitations, considering also a safety threshold δ :

$$\underline{j}l_{min} + \delta < \underline{q} < \underline{j}l_{max} - \delta \quad (4)$$

where $\underline{j}l_{min}, \underline{j}l_{max} \in \mathbb{R}^7$ are the vectors containing the physical minimum and maximum limits of the joints.

3. Time Behavior

The time derivative of the considered control variable is the joints velocities.

$$\dot{\underline{x}} = \dot{\underline{q}} \quad (5)$$

4. Jacobian Matrix

Regarding a single robot, its jacobian for this task is a 7×7 identity matrix. But since we are describing both manipulators together, we have to consider that our system configuration vector looks as shown below:

$$\dot{\underline{y}} = \begin{pmatrix} \dot{\underline{q}}_{left} \\ \dot{\underline{q}}_{right} \end{pmatrix} \in \mathbb{R}^{14} \quad (6)$$

So we are going to use a Jacobian J built as follows:

$$J = \underbrace{\begin{bmatrix} I_7 & 0_{7 \times 7} \end{bmatrix}}_{\text{for the left arm}} \in \mathbb{R}^{7 \times 14} \quad , \quad J = \underbrace{\begin{bmatrix} 0_{7 \times 7} & I_7 \end{bmatrix}}_{\text{for the right arm}} \in \mathbb{R}^{7 \times 14} \quad (7)$$

5. Desired Time Behavior (Reference Rate)

The desired time behavior we want to impose on the control variable is a repulsive reference velocity that activates only when the joints violate the safety threshold δ . The activation of the task will be managed with the activation matrix A , while the check below is needed to switch between the lower and upper limit cases.

$$\dot{\bar{x}}_i = \begin{cases} -\lambda(jl_{min,i} + \delta - q_i) & \text{if } |dist_{min}| < \delta \\ \lambda(jl_{max,i} - \delta - q_i) & \text{if } |dist_{max}| < \delta \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

where $\lambda > 0$ is a scalar control gain, and the distances are defined as $dist_{min} = q_i - jl_{min,i}$ and $dist_{max} = jl_{max,i} - q_i$.

6. Activation Function

To design the activation matrix $A \in \mathbb{R}^{7 \times 7}$ we need to take into account the fact that we are dealing with an inequality task, so the task should be turned on and off based on the current state of the manipulators.

- For the lower joint limit activation factor we need a decreasing bell shaped function:

$$Bell_{dec}(q_i, jl_{min})$$

In this way, the constraint is activated when the joint variable is less than the lower limit plus an activation threshold.

- For the upper joint limit activation factor we need an increasing bell shaped function:

$$Bell_{inc}(q_i, jl_{max})$$

In this way, the joint is constrained when its variable is greater than the upper limit minus an activation threshold.

In conclusion, the activation matrix $A(q)$ is a diagonal matrix where the elements along the main diagonal are defined as follows:

$$A_{ii}(q) = Bell_{dec}(q_i, jl_{min}) + Bell_{inc}(q_i, jl_{max}) \quad (9)$$

2.3 Q3: What is the Jacobian relationship for the Bimanual Rigid Grasping Constraint task? How was the task reference computed?

1. Identify Control Variable

In this case the control variable is the relative pose between the left end-effector (L) and the right one (R).

$$\underline{x} = \underline{x}_L - \underline{x}_R \in \mathbb{R}^6 \quad (10)$$

2. Objective

We want to guarantee that the relative distance and orientation between the two end-effectors remain constant:

$$\bar{\underline{x}} = \underline{x}_L - \underline{x}_R = \text{constant} \quad (11)$$

3. Time Behavior

The time derivative of the considered control variable is the difference of the Cartesian velocities.

$$\dot{\bar{\underline{x}}} = \dot{\underline{x}}_L - \dot{\underline{x}}_R \quad (12)$$

4. Jacobian Matrix

Regarding the single robots, we express Cartesian velocities as a function of joint velocities ($\dot{x} = J\dot{q}$). But since we are describing both manipulators together, the overall Jacobian matrix is the composition of the two Jacobians. So we are going to use a J made as follows:

$$J = \begin{bmatrix} J_L & -J_R \end{bmatrix} \in \mathbb{R}^{6 \times 14} \quad (13)$$

Where the differential relation is:

$$\dot{\tilde{x}} = J_L \dot{q}_L - J_R \dot{q}_R = \begin{bmatrix} J_L & -J_R \end{bmatrix} \begin{pmatrix} \dot{q}_L \\ \dot{q}_R \end{pmatrix} \quad (14)$$

5. Desired Time Behavior (Reference Rate)

The desired time behavior we want to impose to the control variable is a null velocity, since the relative pose must remain constant:

$$\dot{\tilde{x}} = 0 \implies \dot{\underline{x}}_L = \dot{\underline{x}}_R \quad (15)$$

6. Activation Function

In this case the constraint is always active on all spatial degrees of freedom, so the activation matrix is simply an identity matrix:

$$A = I_{6 \times 6} \quad (16)$$

2.4 Q4: Comment the behaviour of the robots, using relevant plots. In particular, show the difference (if any) between the desired object velocity, and the velocities of the two end-effectors in the two cases. Add plots to show that the bimanual manipulation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)

2.5 Bimanual Constraint Performance Analysis

To talk about the performances of the bimanual constraint we can consider the two cases shown below.

In all cases the grasping point and the goal object point are the same written in the assignment request, but in the second case the tool altitude limit of the left robot has been tuned in order to create an asymmetrical constraint among arms. For the left robot we have a minimum tool altitude of 0.3 m instead of 0.15 m, so it's more constrained than the other one. Furthermore, in every case is shown the behaviour of the system when the bimanual constraint is activated or deactivated.

Symmetrical Constraints - Bimanual Constraint On

In this scenario, both robots have the same configuration regarding safety limits (0.15m minimum altitude).

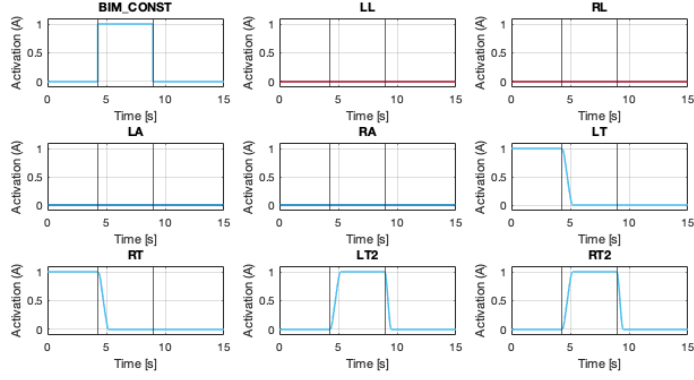


Figure 10: Activation of the tasks during the simulation (Case 1).

By analyzing the activation plot in Figure 10, we can clearly distinguish the three main phases of the operation:

- **Approach** ($t < 4.5s$): The tool tasks LT and RT are high (1), guiding the robots towards the object.
- **Transport** ($4.5s < t < 9s$): The bimanual constraint task switches to 1, activating the rigid constraint between the arms. Simultaneously, LT2 and RT2 (move object) become active.
- **Final** ($t > 9s$): The motion and joint limits tasks are deactivated.

It is important to notice that the safety tasks regarding the altitude (LA, RA) and the joint limits (LL, RL) remain at 0 for the entire duration. This implies that the standard trajectory was safe enough and did not require any repulsive intervention from the safety layer.

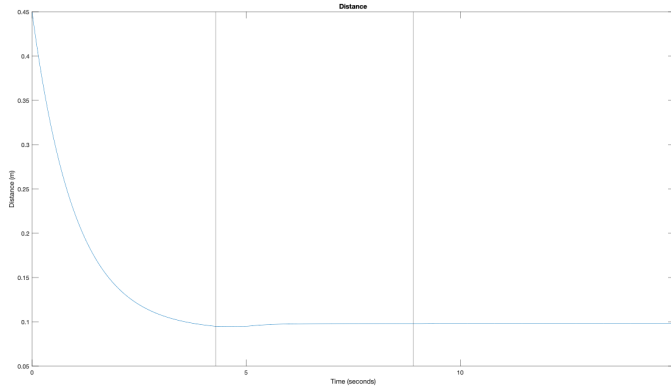


Figure 11: Relative distance between the two end-effectors (Case 1).

Figure 11 shows the norm of the distance vector between the two end-effectors. During the first phase, the distance decreases as the robots approach the grasping points. Once the bimanual constraint is activated ($t \approx 4.5s$), the distance becomes perfectly constant. This visually confirms the correctness of the bimanual constraint: the relative position between the left and right hand is locked, simulating a rigid grasp of the object preventing it from falling or being compressed.

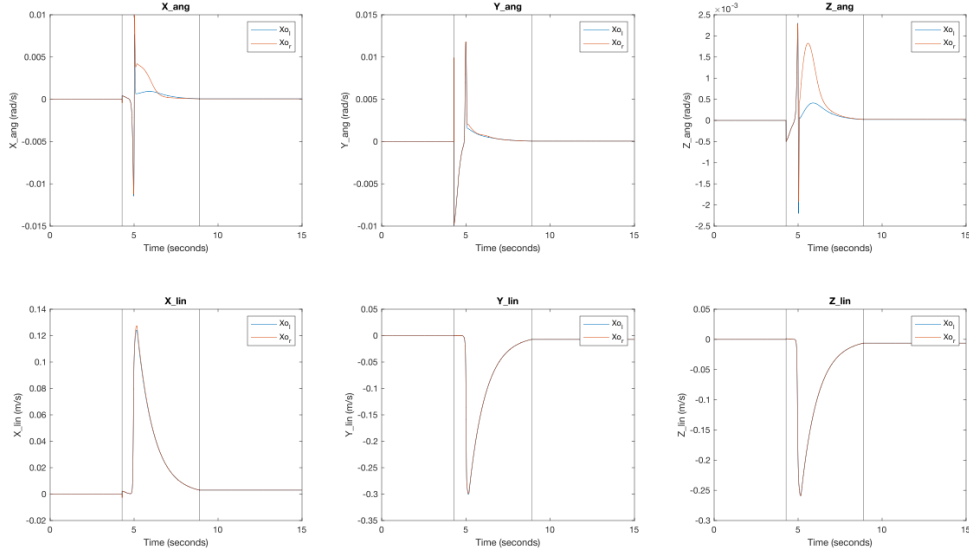


Figure 12: Relative velocities between the end-effectors (Case 1).

Finally, Figure 12 displays the relative velocities (linear and angular) imposed by the bimanual task. We can observe a transient spike exactly at the switching instant ($t \approx 4.5s$). This is due to small misalignments between the actual relative pose and the desired one at the moment of activation. The controller quickly compensates for this error, driving the relative velocities to zero, which ensures the maintenance of the rigid constraint for the rest of the transport phase.

Symmetrical Constraints - Bimanual Constraint Off

In this test, the simulation is repeated with the same symmetrical configuration, but the bimanual constraint task has been deactivated.

The activation plot is not reported here because it appears identical to Figure 10. The State Machine still transitions between sets at the same timestamps, and the safety tasks (LA, RA) remain inactive since the trajectory is safe.

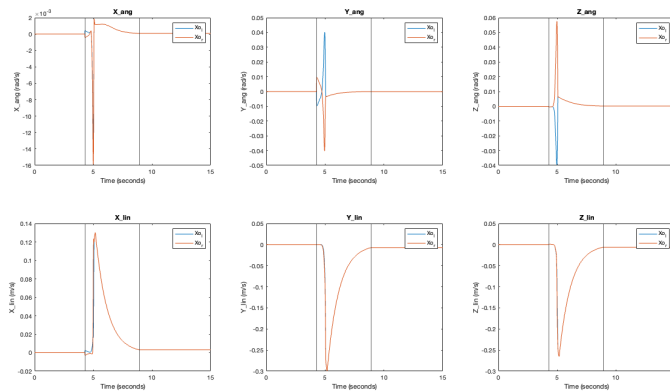


Figure 13: Relative velocities with bimanual constraint deactivated (Case 1).

Figure 13 shows the relative velocities. Since the trajectory does not conflict with any safety task, the motion remains smooth and very similar to the constrained case. This is because the reference generator provides synchronized target poses for both arms; effectively, the robots are independently tracking two consistent trajectories.

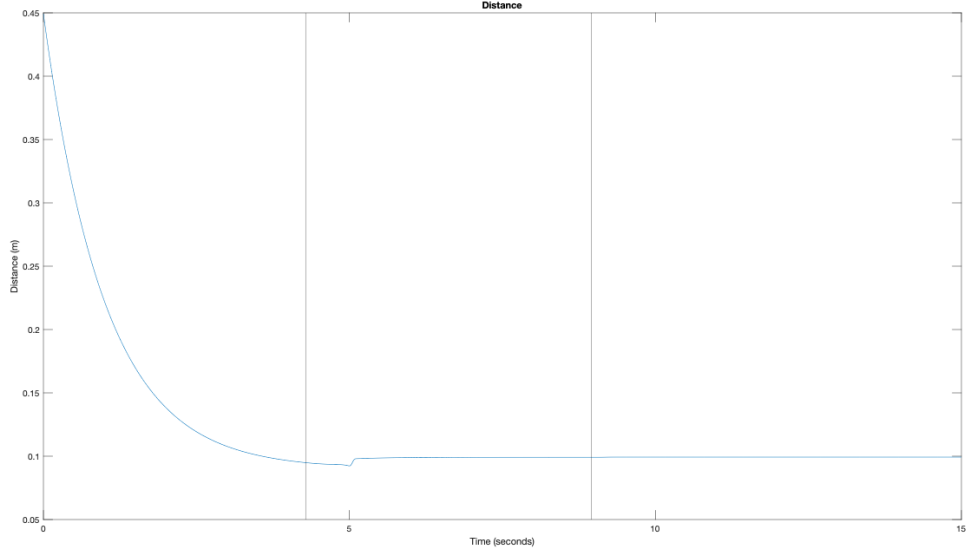


Figure 14: Relative distance with bimanual constraint deactivated (Case 1).

The difference, however, is noticeable in Figure 14. Unlike the previous case where the distance was perfectly constant after the grasp ($t > 4.5s$), here we observe a slight variation (a small step) in the distance profile immediately after the grasping phase. Without the active bimanual constraint forcing the relative error to zero, small synchronization delays or numerical discrepancies between the two independent arm controllers cause the distance to drift slightly from the nominal value.

Case 2: Asymmetrical Constraints - Bimanual Constraint On

In this specific test, the bimanual constraint is active. This allows us to observe how the cooperative system behaves when a safety limit is triggered on only one of the agents.

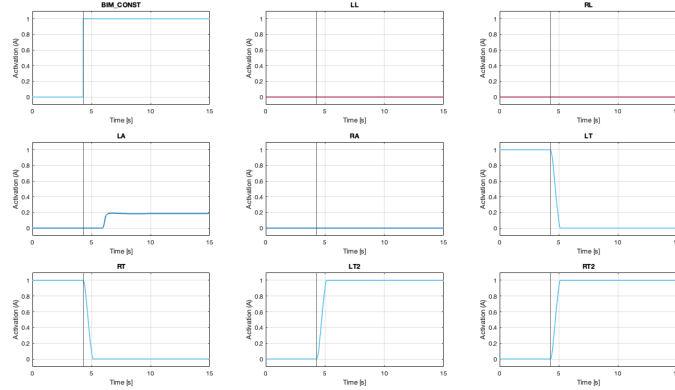


Figure 15: Activation of the tasks with asymmetrical constraints (Case 2).

The activation plot in Figure 15 highlights the main difference compared to the symmetrical case. While the right arm's altitude safety task (RA) remains inactive (at 0) because its limit is set to 0.15m, the left arm's safety task (LA) clearly activates during the transport phase. This happens because the reference trajectory tries to push the object below 0.3m. Since the left arm is strictly constrained to 0.3m, its safety layer intervenes to prevent the violation.

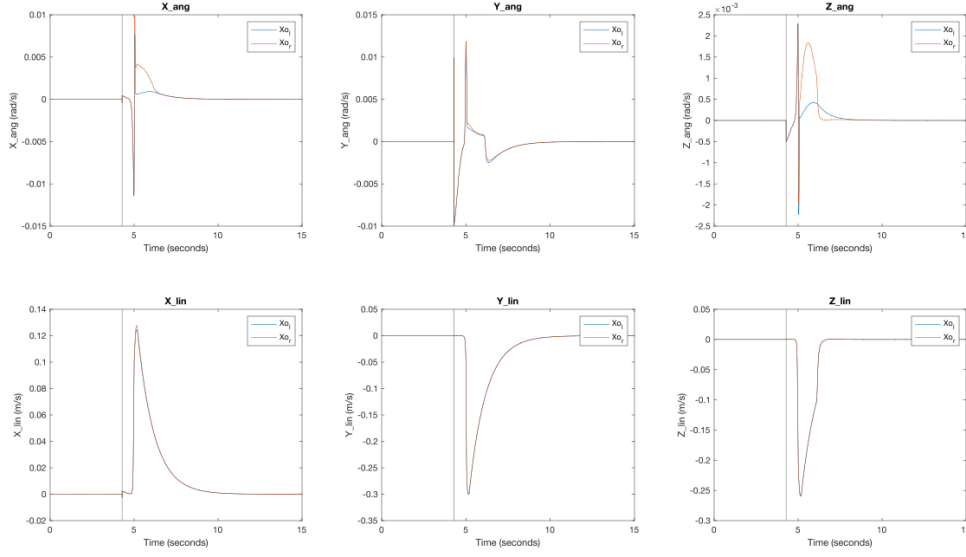


Figure 16: Relative velocities with asymmetrical constraints (Case 2).

The effect of this activation is visible in the velocity profile shown in Figure 16. When the LA task activates, it imposes a zero vertical velocity on the left end-effector to stop its descent. However, due to the presence of the highest-priority bimanual constraint task, this constraint propagates to the entire system. We can observe that the relative velocities converge to zero and remain stable, meaning the system successfully handles the conflict between the motion command (which wants to go down) and the safety command (which forces a stop). It is needed to specify that, in this case, the bimanual constraint remains active until the end of the simulation, since the object never reach its goal.

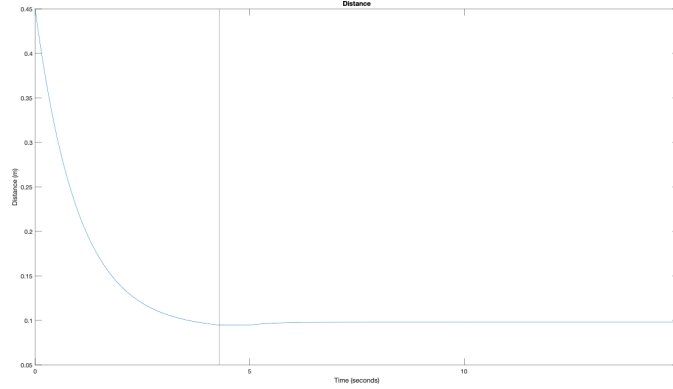


Figure 17: Relative distance with asymmetrical constraints (Case 2).

The most significant result is shown in Figure 17. Despite the left arm being blocked by its safety limit, the relative distance between the two hands remains perfectly constant throughout the task. This implies a crucial cooperative behavior: although the right arm had not reached its own physical limit (0.15m) and could theoretically continue moving downwards, it was forced to halt its descent to satisfy the rigid link with the left arm. The bimanual constraint effectively couples the safety limits: the most restrictive constraint on one arm automatically limits the workspace of the other arm to preserve the integrity of the grasp.

Case 2: Asymmetrical Constraints - Bimanual Constraint Off

Finally, the system is tested with asymmetrical constraints (0.3m for Left, 0.15m for Right) but with the bimanual constraint deactivated. This represents the worst-case scenario where safety limits are present but there is no cooperative layer to synchronize the agents.

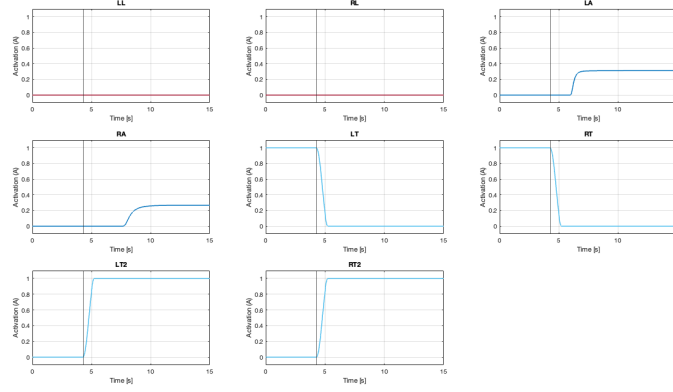


Figure 18: Activation of the tasks with asymmetrical constraints and deactivated bimanual task (Case 2 OFF).

The activation plot in Figure 18 tells the whole story. We can observe that the Left Arm's safety task (LA) activates first (at $t \approx 4.5s$) because it hits the restrictive 0.3m limit. Crucially, the Right Arm's safety task (RA), which is less constrained (0.15m), remains inactive at that moment. Since the BIM.CONST is off, there is no link forcing the Right Arm to stop. Consequently, the Right Arm continues to move downwards and its safety task (RA) activates only much later (at $t \approx 7s$), when it finally reaches its own lower limit.

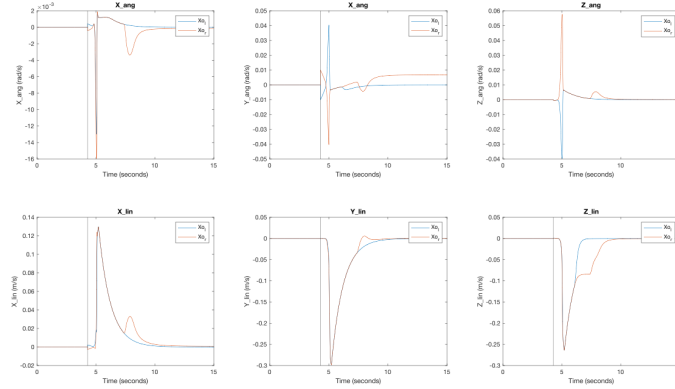


Figure 19: Relative velocities with asymmetrical constraints and deactivated bimanual task (Case 2 OFF).

Figure 19 confirms this decoupled behavior. When the left arm stops (velocity goes to zero due to LA), the right arm's velocity continues to follow the reference trajectory. This creates a large divergence in the relative velocity profile, which is no longer regulated to be zero.

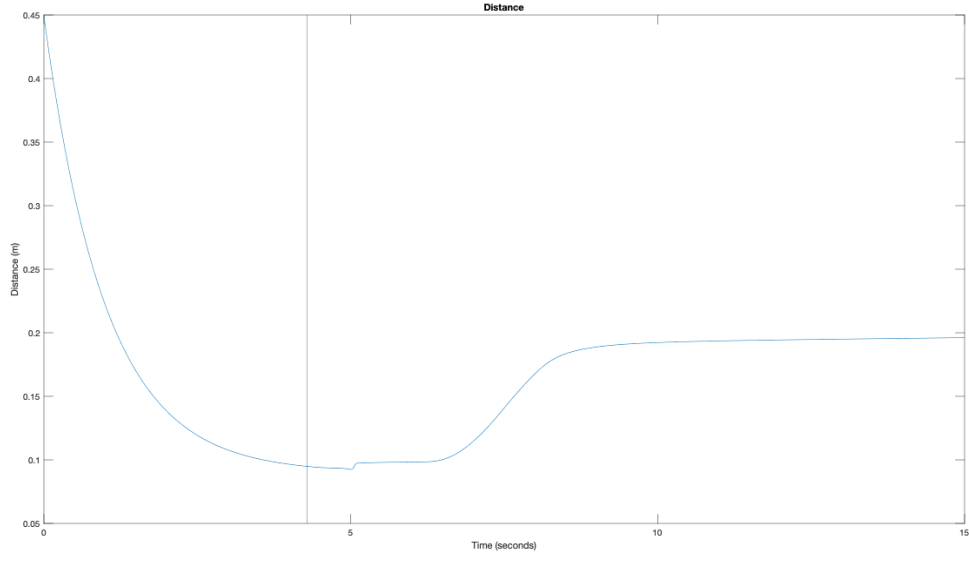


Figure 20: Relative distance with asymmetrical constraints and deactivated bimanual task (Case 2 OFF).

The physical consequence is shown in Figure 20. As soon as the left arm stops while the right arm continues descending, the relative distance between the hands increases drastically (the curve shoots up).

3 Exercise 3: Cooperative manipulation

In this exercise, it is required to perform cooperative manipulation by implementing the task priority algorithm considering the two Franka Panda manipulators as two distinct robots.

1. The first phase foresees to move the tool frames to the grasping points, by implementing the “move to” action for both manipulators. **Please note: each manipulator has his own Task Priority Inverse Kinematic Algorithm.** Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT:** the position of the grasping points can be computed by knowing the origin of the object frame wO_o and the object length l_{obj} . The goal orientation of the tool frames is obtained by rotating the tool frames 20 deg around their y-axis.

$${}^wO_o = [0.5, 0, 0.59]^T (m); \quad (17)$$

$$l_{obj} = 6 \text{ (cm)}. \quad (18)$$

During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

2. Once the manipulators reach the grasping points the second phase of the mission should start. Implement the *Cooperative Rigid Constraint* task to carry the object as a rigid body.
 - (a) Define the object frame as a rigid body attached to the tool frame of each manipulator.
 - (b) Define the rigid grasp task.
 - (c) You have to move the object to another position while both manipulators hold it firmly. The desired object goal position is

$${}^wO_g = [0.60, 0.40, 0.48]^T (m) \quad (19)$$

- (d) Compute the *non-cooperative* object frame velocities. **HINT:** Suppose manipulators communicate ideally and can exchange the respective end-effector velocities
- (e) Apply the coordination policy to the *non-cooperative* object frame velocities.
- (f) Compute the *cooperative* object frame velocities.

Note that the transition for the *Cooperative Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

3. Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Again, test it twice, once with the provided (reachable) goal, and then with a goal or with a different grasp configuration that triggers the activation of multiple joint limits or a joint limit and minimum altitude by one manipulator. The idea is that this second position should trigger the cooperation policy (i.e., the cooperative velocity of one manipulator should be different than the original desired object velocity, and that the cooperative velocity is very similar to its non cooperative one, showing that the other robot adapts).

3.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action. Report clearly which action is used by TPIK1 and TPIK2 in the cooperative algorithm.

Since each robot should work separately, they have separate action managers and task sets. We have `go_to_grasp_set`, `move_grasped_obj_set`, `final_set`, and `unified_set`, one for each robot.

`go_to_grasp_set` is the first action to be performed, in order to grasp the object. It is composed of `joint_limit_task`, which is a safety task, `min_altitude`, also a safety task, and `tool_task`, which is the action-defining task and has the lowest priority.

`move_grasped_obj_set` is the second action, performed when both robots have grasped the object. This set includes `coop_constraint_task`, which has the highest priority, `joint_limit_task`, `min_altitude`, and `tool_task_2`.

`final_set` is, as the name suggests, the last action, which only requires `min_altitude` to remain active.

`unified_set` is the actual task set used by the action manager. It contains `coop_constraint_task`, `joint_limit_task`, `min_altitude`, `tool_task`, and `tool_task_2`.

When computing ICAT, the `unified_set` is compared with the current action and the last action performed, in order to determine which tasks should be activated, deactivated, or remain inactive.

After grasping, the robots should be linked together. We force `coop_constraint_task` to be inactive when computing TPIK1 by using a flag, but it is activated at the end of the `computeICAT` procedure. At the same time, `tool_task_2` is deactivated. In this way, `coop_constraint_task` is active and has the highest priority when computing TPIK2.

Table 3: Hierarchy table

Task	Type	Attribute	Go to Grasp	Move Grasped Obj	Final	Unified
Cooperative Constraint	E	Constraint		1		1
Joint Limits	I	Safety	1	2		2
Minimum Altitude	I	Safety	2	3	1	3
Tool Task	E	Action Defining	3			4
Tool Task 2	E	Action Defining		4		5

The transition is handled by the two `StateMachine` of the arms, which share status information with each other. By updating the state machines, `dist_to_goal` and `rot_to_goal` of both arms are compared with a threshold.

1. **Cooperative Constraint Task (Equality):** Enforces the cooperative manipulation constraint by using the feasible cooperative velocities, that come out from the coordination policy, while the object is grasped. Formulated as an equality constraint with the highest priority, this task strictly synchronizes both arms' motion to preserve the grasp geometry and load distribution during cooperative manipulation. This task is shared between both arms, coupling their motion to achieve unified object transport.
2. **Joint Limit Task (Inequality):** Enforces kinematic safety boundaries by preventing manipulator joints from exceeding their mechanical limits during all operational phases. Formulated as an inequality constraint, this task activates only when joints approach their physical bounds, allowing free motion within the safe workspace while intervening to prevent damage or singularities near the limits.
3. **Minimum Altitude (Inequality):** Maintains a minimum vertical clearance from the seabed to prevent collisions during dual-arm manipulation and object transport maneuvers, ensuring safe operation throughout all action phases. As an inequality task, it activates only when the vehicle approaches the minimum threshold, allowing greater altitudes without penalization while guaranteeing obstacle avoidance.
4. **Tool Task (Equality):** Drives the end-effector (tool frame) of each manipulator arm to the object grasp position in 3D space during the initial approach phase, executing the primary objective of reaching and acquiring the target object. The equality constraint ensures precise positioning of each tool at the designated grasp location with minimal error, enabling successful bilateral grasping.
5. **Tool Task 2 (Equality):** Controls the end-effector trajectories of both manipulator arms to execute the coordinated object transport motion after successful grasping, driving the grasped object along the desired path toward the target location. The equality formulation ensures continuous tracking of the reference trajectory with both arms moving in concert.

3.2 Q2: Comment the behaviour of the robots, using relevant plots. In particular, make sure there are differences between the desired object velocity and the non-cooperative Cartesian velocities at least in one simulation. Show also how the cooperative velocities of the two end-effectors behave. Add plots to show that the cooperation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)

To comprehensively evaluate the cooperative manipulation system, we conducted experiments in two distinct scenarios:

1. **Baseline Scenario:** The standard configuration provided in the assignment, where both manipulator arms operate with identical constraints and task priorities.
2. **Modified Scenario:** An identical setup where one of the two arms (specifically, the left arm) is subject to a more restrictive minimum altitude constraint, creating asymmetric operational conditions that test the system's ability to maintain cooperation under unequal constraints.

For each scenario, we generated five key plots to analyze different aspects of the cooperative behavior:

1. **Distance Between the Two Arms (Tool Frames)** This plot monitors the Euclidean distance between the right and left end-effector frames throughout the manipulation task. This is the most critical metric for verifying cooperative behavior: a constant distance demonstrates that the cooperative constraint is properly satisfied, confirming that both arms maintain their relative configuration while moving the grasped object.

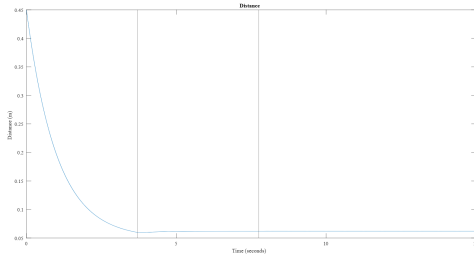


Figure 21: Baseline Scenario

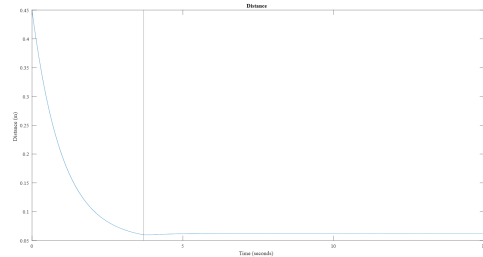


Figure 22: Modified Scenario

As we expect, there is no variation in this behavior in the two scenario, since the cooperative manipulation is respected in both.

2. **Left Arm Activation** This plot displays the activation level of the left arm's tasks over time, providing insight into the task sequencing and priority management.

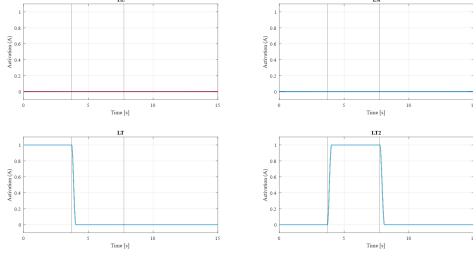


Figure 23: Baseline Scenario

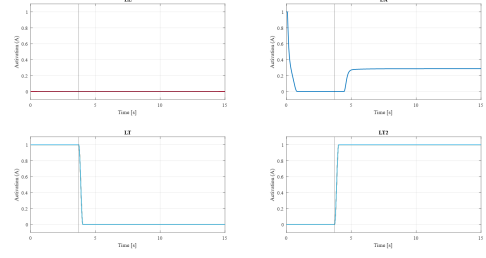


Figure 24: Modified Scenario

We can see here, in the modified scenario, that the minimum altitude task has been activated for the left arm. Also, the left arm never reach the target and `tool_task_2` never stops.

3. **Right Arm Activation** Analogous to the left arm activation, this plot displays the activation level of the right arm's tasks over time, providing insight into the task sequencing and priority management.

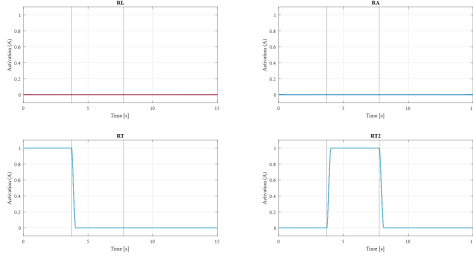


Figure 25: Baseline Scenario

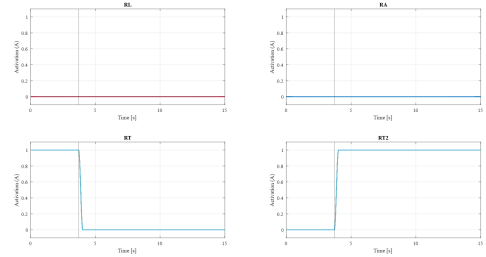


Figure 26: Modified Scenario

Here we see, in the modified scenario, only that the right arm never reach the target, so `tool_task_2` never stops.

4. Difference Between Individual Arm Velocities and Agreed Cooperative Velocity It shows the difference between:

- The Cartesian velocity each arm would execute if acting independently (non-cooperative velocity)
- The agreed-upon cooperative velocity that both arms should follow to maintain the bimanual constraint

Significant differences in this plot indicate that the individual arm controllers are being overridden by the cooperation constraint, which is the expected and desired behavior. This is particularly evident when one arm faces additional constraints (as in the modified scenario), forcing the cooperative controller to mediate between conflicting objectives.

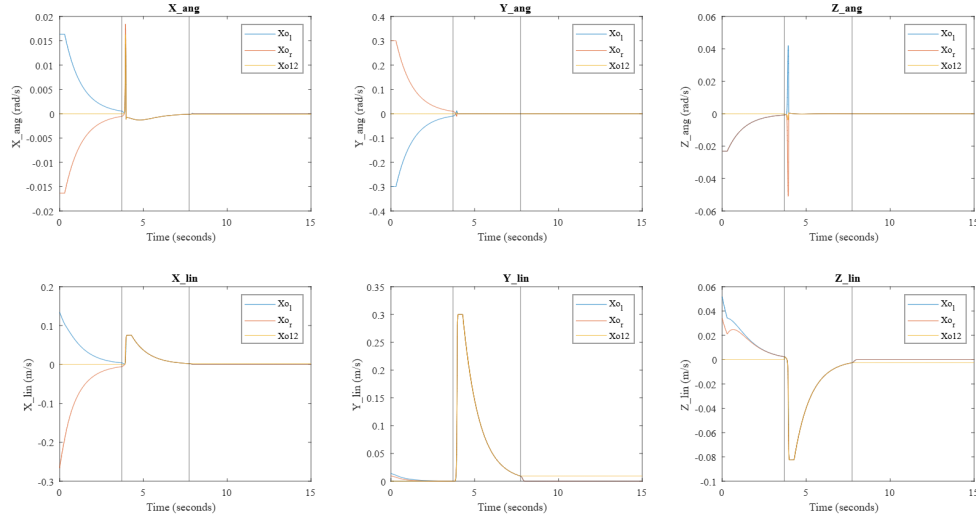


Figure 27: Baseline Scenario

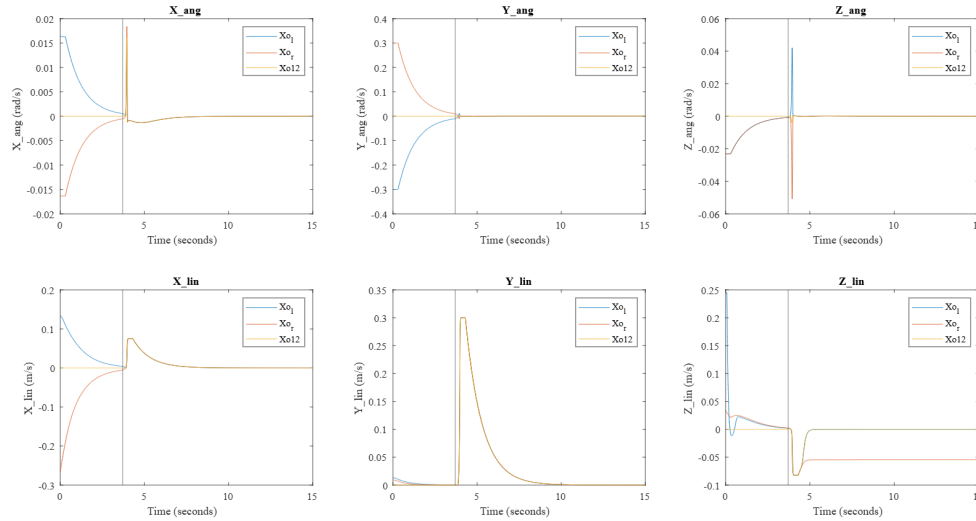


Figure 28: Modified Scenario

We can clearly see, in the modified scenario, the difference between the non-cooperative velocities of the two arms (mainly in Z_{lin}), and how the cooperative velocity follows the left arm velocity, since it is the most constrained.

5. **Difference Between Agreed Cooperative Velocity and Actual Arm Velocities** This plot verifies the execution fidelity of the cooperative control system by comparing:

- The cooperative velocity computed by the cooperative constraint task
- The actual velocities executed by each arm's end-effector

This difference should be zero or negligible, confirming that both arms successfully track the agreed-upon cooperative velocity, but this only when the object is performing the action `move_grasped_obj`.

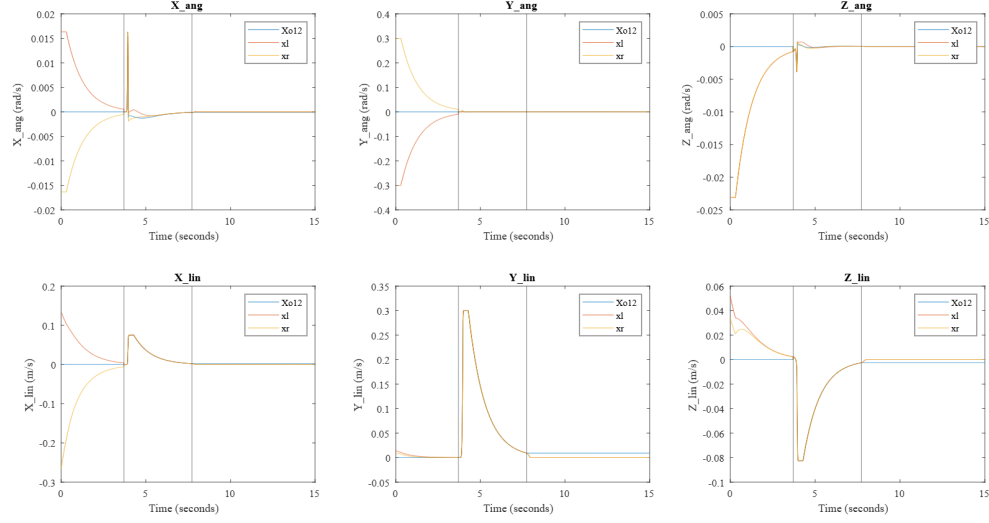


Figure 29: Baseline Scenario

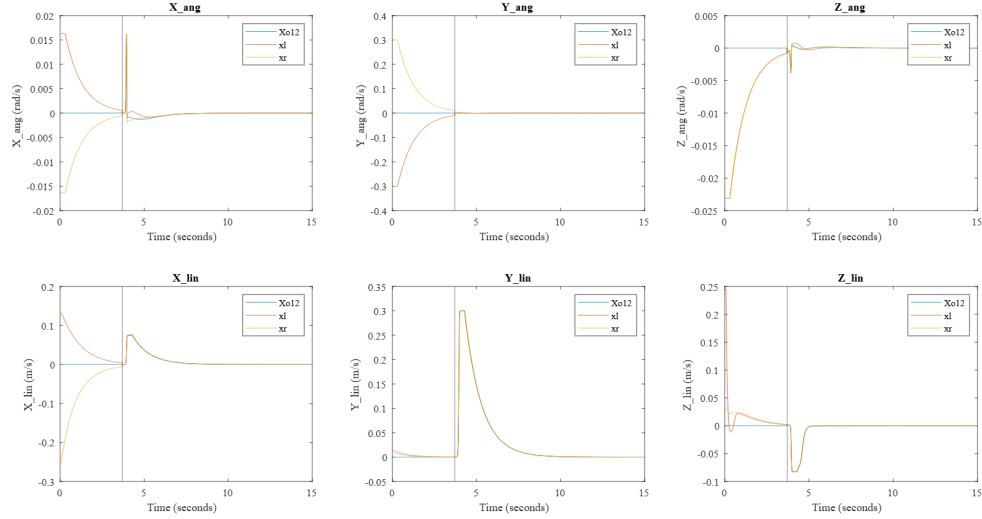


Figure 30: Modified Scenario

This shows that the cooperative manipulation is working because, even if the right arm do not have constraint from his side, he agree with the feasible velocity.

The comparison between the two scenarios reveals:

1. **Robustness of Cooperative Control:** Even under asymmetric constraints, the system maintains the cooperative grasp (constant distance) while adapting the cooperative trajectory.

2. **Conflict Resolution:** Plot 4 in the modified scenario quantifies how much individual arm controllers must “compromise” to maintain cooperation, with the more constrained arm (left) showing larger deviations from its preferred motion.
3. **Task Priority Effectiveness:** The high priority of the `coop_constraint_task` ensures cooperation is never sacrificed, even when individual arms face conflicting objectives.